



**Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey**

Escuela de Ingeniería y Ciencias

Ingeniería en Ciencias de Datos y Matemáticas

Uso de álgebras modernas para seguridad y criptografía, MA2006B.604

SDK - Etapa 1: Gestor de Identidades

Equipo 2:

Ana Lidia Hernández Díaz	A00838643
Ana Paula García Valverde	A01174572
María Fernanda Montoya López	A01743214
Javier Rojas Orrante	A01352213
Xavier Lago Hicks	A01403177

Profesores:

Raúl Gómez Anas Wajid Alberto F. Martínez

Proyecto: Firmas Solidarias – Tecnología Criptográfica para los Derechos Humanos
Socio Formador: Casa Monarca

Monterrey, Nuevo León. 30 de abril de 2026

Índice general

1. Descripción General	4
1.1. Propósito del sistema	4
1.2. Alcance	4
1.3. Casos de uso principales	4
2. Arquitectura del Sistema	5
2.1. Tipo de arquitectura	5
2.2. Componentes principales	5
2.3. Diagrama general	5
3. Stack Tecnológico	6
3.1. Lenguajes	6
3.2. Framework	6
3.3. Bibliotecas principales	6
4. Estructura del Proyecto	7
4.1. Organización de carpetas	7
4.2. Descripción de módulos	7
5. Configuración del Entorno	9
5.1. Requisitos de software	9
5.2. Variables de entorno	9
5.3. Archivos de configuración	9
6. Instalación y Ejecución	10
6.1. Pasos para levantar el sistema	10
6.2. Comandos principales	10
7. Base de Datos	11
7.1. Modelo de datos	11
7.2. Tablas principales	11
7.3. Relaciones	11
8. API / Interfaces	12
8.1. Endpoints JSON (API)	12
8.2. Rutas HTML principales	12

8.3. Rutas de gestión de usuarios (UI)	12
8.4. Rutas de beneficiarios y exportación	13
8.5. Parámetros de ejemplo	13
9. Flujos y Lógica de Negocio	14
9.1. Procesos clave	14
9.1.1. Ciclo de vida de una cuenta	14
9.1.2. Autenticación criptográfica (challenge-response)	14
9.1.3. Sesión y cookies	14
9.1.4. Expiración automática	14
9.2. Reglas importantes	14
9.3. Casos críticos	15
10. Ejemplos de Uso	16
10.1. Caso 1: Crear y activar un coordinador	16
10.2. Caso 2: Iniciar sesión con certificado	16
10.3. Caso 3: Revocar un usuario	16
10.4. Caso 4: Exportar auditoría	16
11. Manejo de Errores	17
11.1. Estrategia de errores	17
11.2. Mensajes comunes	17
11.3. Registro (logs)	17
12. Pruebas	18
12.1. Tipos de pruebas	18
12.2. Pruebas funcionales manuales	18
13. Despliegue	19
13.1. Despliegue con Docker	19
13.2. Despliegue en Render	19
13.3. Entornos	19
13.4. Consideraciones	19
14. Seguridad	20
14.1. Autenticación	20
14.2. Manejo de datos sensibles	20
14.3. Buenas prácticas implementadas	20

15. Guía de Contribución	21
15.1. Convenciones de código	21
15.2. Flujo de trabajo con Git	21
15.3. Cómo agregar funcionalidades	21
16. Problemas Conocidos	22
16.1. Bugs actuales	22
16.2. Limitaciones	22
17. FAQ Técnica	23
18. Referencias	24

1. Descripción General

1.1. Propósito del sistema

El Gestor de Identidades es una aplicación web diseñada para Casa Monarca que centraliza la administración de cuentas de usuario dentro de la organización. Su objetivo principal es garantizar que cada persona que accede al sistema esté debidamente autenticada y autorizada según su rol, empleando mecanismos de criptografía de llave pública para los roles que manejan información sensible.

El sistema cubre tres necesidades operativas:

- Control de acceso basado en roles con cuatro niveles de privilegio.
- Emisión y gestión de certificados digitales X.509 para roles privilegiados.
- Registro de auditoría de todas las acciones realizadas en la plataforma.

1.2. Alcance

El sistema permite:

- Crear, activar, revocar y eliminar cuentas de usuario.
- Asignar y cambiar roles (Administrador, Coordinador, Operativo, Voluntario).
- Generar pares de llaves RSA-2048 y certificados X.509 para roles que lo requieran.
- Autenticación por contraseña (PBKDF2) para roles básicos y por llave privada + certificado para roles privilegiados.
- Exportar reportes PDF con hash de integridad SHA-256.
- Registrar beneficiarios vinculados a las áreas de atención.
- Mecanismo de recuperación mediante administrador espejo.

Limitaciones actuales:

- No se implementa comunicación por correo electrónico para notificaciones ni recuperación de contraseña.
- No se cuenta con un módulo de gestión documental completo.
- La base de datos en modo desarrollo es SQLite; para producción se requiere PostgreSQL.

1.3. Casos de uso principales

1. **Registro y activación de usuarios:** El administrador crea una cuenta, define el rol y la activa. Si el rol requiere criptografía, se genera el material criptográfico.
2. **Inicio de sesión criptográfico:** El usuario sube su llave privada (`private_key.pem`) y su certificado (`certificate.pem`), ingresa su contraseña, y el servidor ejecuta un challenge-response con firma RSA-PSS.
3. **Revocación de emergencia:** El administrador revoca el acceso de un usuario de forma inmediata, invalidando su contraseña y sus credenciales.
4. **Recuperación por espejo:** Si el administrador principal pierde acceso, se activa la cuenta de respaldo con material criptográfico independiente.
5. **Consulta de auditoría:** El administrador revisa el registro cronológico de eventos del sistema para fines de trazabilidad.

2. Arquitectura del Sistema

2.1. Tipo de arquitectura

La aplicación sigue una arquitectura monolítica de tres capas, contenida en un solo proceso de Python:

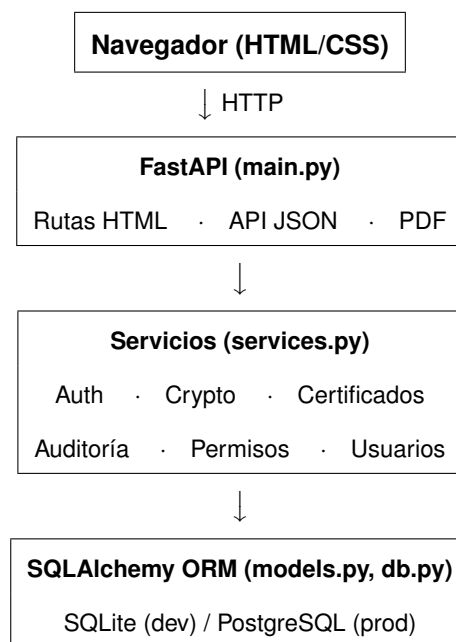
1. **Capa de presentación:** Vistas HTML generadas dinámicamente con Python (f-strings). No se usa un motor de plantillas externo como Jinja2; todo el HTML se construye dentro de `main.py`.
2. **Capa de lógica de negocio:** Servicios en `services.py` que encapsulan la lógica de autenticación, criptografía, certificados, permisos y auditoría.
3. **Capa de datos:** Modelos ORM en `models.py` con SQLAlchemy 2.0. La conexión se configura en `db.py`.

2.2. Componentes principales

- FastAPI como framework HTTP con rutas HTML y API JSON.
- SQLAlchemy 2.0 como ORM para la persistencia.
- Módulo `cryptography` de Python para operaciones RSA, X.509 y AES.
- `itsdangerous` para la firma HMAC de cookies de sesión.
- `fpdf2` para la generación de reportes PDF.

2.3. Diagrama general

El sistema se organiza en cuatro capas que se comunican de arriba hacia abajo:



3. Stack Tecnológico

3.1. Lenguajes

- **Python 3.11+** — Lenguaje principal del backend y generación de vistas.
- **HTML/CSS** — Interfaz de usuario renderizada en el servidor.
- **JavaScript** (mínimo, inline) — Interacciones puntuales en el frontend.
- **SQL** — Esquema de referencia en `sql/schema.sql`.

3.2. Framework

- **FastAPI $\geq$ 0.115.0** — Framework web asíncrono con tipado estricto y documentación automática OpenAPI.

3.3. Bibliotecas principales

Biblioteca	Versión mín.	Uso
cryptography	43.0.0	Generación de llaves RSA, certificados X.509, cifrado AES-256, firmas digitales.
SQLAlchemy	2.0.35	ORM para la capa de persistencia.
itsdangerous	2.2.0	Firma HMAC de cookies de sesión (URLSafeTimedSerializer).
uvicorn	0.30.0	Servidor ASGI para ejecutar FastAPI.
fpdf2	2.7.0	Generación de reportes PDF.
python-multipart	0.0.9	Lectura de formularios multipart/form-data (subida de archivos).
pydantic-settings	2.5.2	Carga y validación de variables de entorno.
psycopg	3.2.0	Driver PostgreSQL para producción.

4. Estructura del Proyecto

4.1. Organización de carpetas

```
RetoCriptografia-Equipo2/
|-- app/
| |-- __init__.py # Marca el paquete Python
| |-- config.py # Variables de configuracion (pydantic-settings)
| |-- db.py # Conexion y sesion de base de datos
| |-- deps.py # Dependencias inyectadas en FastAPI
| |-- main.py # Rutas, vistas HTML y logica de presentacion
| |-- models.py # Modelos ORM (User, Role, AuditLog, etc.)
| |-- schemas.py # Esquemas Pydantic para respuestas JSON
| |-- services.py # Logica de negocio y criptografia
| +-- static/ # Archivos estaticos (CSS, imagenes)
|-- docs/ # Documentacion del proyecto
|-- generated/
| +-- certs/ # Certificados y llaves generados
| |-- ca/ # Certificado y llave de la CA
| +-- users/ # Material criptografico por usuario
|-- sql/
| +-- schema.sql # Esquema SQL de referencia
|-- .env.example # Plantilla de variables de entorno
|-- Dockerfile # Imagen Docker
|-- pyproject.toml # Metadatos y dependencias
+-- render.yaml # Configuracion de despliegue en Render
```

4.2. Descripción de módulos

Módulo	Responsabilidad
config.py	Define la clase Settings que carga y valida las variables de entorno (DATABASE_URL, SESSION_SECRET, CERTS_DIR, entre otras). Detecta si el entorno es producción y aplica validaciones adicionales.
db.py	Crea el engine de SQLAlchemy y la fábrica de sesiones SessionLocal. Diferencia la configuración entre SQLite y PostgreSQL.
deps.py	Provee el generador get_db() que inyecta una sesión de base de datos en cada endpoint de FastAPI.
main.py	Contiene todas las rutas HTTP, la generación de HTML, la lógica de sesión con cookies firmadas y la exportación de PDFs. Es el punto de entrada de la aplicación.
models.py	Define los modelos ORM: User, Role, Permission, RolePermission, AuditLog, SystemSecret, Notification y Beneficiario.
schemas.py	Esquemas Pydantic (UserOut, MeOut, AuditLogOut, RoleOut) para serializar respuestas de la API JSON.

Módulo	Responsabilidad
<code>services.py</code>	Concentra la lógica de negocio: hashing de contraseñas (PBKDF2), emisión de certificados X.509, autenticación con firma digital, gestión de roles, auditoría, expiración de cuentas y administrador espejo.

5. Configuración del Entorno

5.1. Requisitos de software

- Python 3.11 o superior.
- pip (incluido con Python).
- Git.
- Docker (opcional, para despliegue con contenedor).
- PostgreSQL 14+ (solo para producción).

5.2. Variables de entorno

Las variables se definen en un archivo `.env` en la raíz del proyecto. El archivo `.env.example` sirve como plantilla:

Variable	Valor por defecto	Descripción
APP_NAME	gestor-identidades-demo	Nombre de la aplicación.
APP_HOST	127.0.0.1	Host de escucha.
APP_PORT	8000	Puerto de escucha.
ENVIRONMENT	development	development o production.
DATABASE_URL	sqlite:///./identity_demo.db	Cadena de conexión a la base de datos.
CERTS_DIR	./generated/certs	Directorio para certificados generados.
SESSION_SECRET	(inseguro por defecto)	Secreto para firmar cookies. En producción debe ser aleatorio y seguro.
SESSION_COOKIE_SECURE	false	true en producción (solo HTTPS).
SEED_DEMO_DATA	true	Carga usuarios y beneficiarios de demostración.

Para producción se agregan:

- `BOOTSTRAP_ADMIN_FULL_NAME` — Nombre del administrador inicial.
- `BOOTSTRAP_ADMIN_EMAIL` — Correo del administrador inicial.
- `BOOTSTRAP_ADMIN_PASSWORD` — Contraseña del administrador inicial.

5.3. Archivos de configuración

- `.env` — Variables de entorno (no se sube al repositorio).
- `.env.example` — Plantilla de referencia.
- `pyproject.toml` — Metadatos del proyecto y lista de dependencias.
- `Dockerfile` — Definición de la imagen Docker.
- `render.yaml` — Configuración declarativa para despliegue en Render.

6. Instalación y Ejecución

6.1. Pasos para levantar el sistema

1. Clonar el repositorio:

```
git clone https://github.com/<usuario>/RetoCriptografia-Equipo2.git
cd RetoCriptografia-Equipo2
```

2. Crear un entorno virtual:

```
python -m venv venv
```

3. Activar el entorno virtual:

```
# Linux / macOS
source venv/bin/activate

# Windows
venv\Scripts\activate
```

4. Instalar las dependencias:

```
pip install .
```

5. Copiar y ajustar las variables de entorno:

```
cp .env.example .env
```

6. Iniciar el servidor:

```
uvicorn app.main:app --reload
```

7. Abrir `http://127.0.0.1:8000` en el navegador.

En modo demo, las credenciales del administrador son `admin / admin`.

6.2. Comandos principales

Comando	Descripción
<code>uvicorn app.main:app -reload</code>	Inicia el servidor en modo desarrollo con recarga automática.
<code>pip install .</code>	Instala el proyecto y sus dependencias a partir de <code>pyproject.toml</code> .
<code>docker build -t gestor .</code>	Construye la imagen Docker.
<code>docker run -p 8000:8000 gestor</code>	Ejecuta el contenedor en el puerto 8000.

7. Base de Datos

7.1. Modelo de datos

La base de datos se crea automáticamente al iniciar la aplicación. SQLAlchemy genera las tablas a partir de los modelos definidos en `models.py`. Adicionalmente, el servicio `SchemaService` ejecuta migraciones incrementales para agregar columnas que no existan (`ALTER TABLE`).

7.2. Tablas principales

Tabla	Descripción
<code>users</code>	Usuarios del sistema. Almacena credenciales, estado de la cuenta, certificado PEM, llave pública PEM, llave privada cifrada, serial del certificado, fechas de vigencia, hash de contraseña, intentos de login y si es administrador espejo.
<code>roles</code>	Catálogo de roles (ADMIN, COORDINADOR, OPERATIVO, VOLUNTARIO).
<code>permissions</code>	Catálogo de permisos definidos como pares (<code>resource</code> , <code>action</code>).
<code>role_permissions</code>	Tabla pivote que asocia roles con permisos.
<code>audit_logs</code>	Registro de auditoría: tipo de evento, actor, objetivo, acción, recurso, resultado, metadatos, IP y user-agent.
<code>system_secrets</code>	Almacén de secretos del sistema (llave privada de la CA, llave privada del admin firmante).
<code>notifications</code>	Notificaciones generadas automáticamente (certificados próximos a vencer, cuentas bloqueadas, etc.).
<code>beneficiarios</code>	Personas atendidas por Casa Monarca, con datos de área, estatus y notas.

7.3. Relaciones

- `users.role_id → roles.id` — Cada usuario pertenece a un rol.
- `role_permissions.role_id → roles.id` — Cada entrada asocia un rol con un permiso.
- `role_permissions.permission_id → permissions.id` — Referencia al permiso.
- `beneficiarios.created_by_user_id → users.id` — Qué usuario registró al beneficiario.
- `notifications.user_id → users.id` — Notificación dirigida a un usuario.

8. API / Interfaces

El sistema expone dos tipos de interfaces: rutas HTML que renderizan la interfaz gráfica y endpoints JSON para consulta programática.

8.1. Endpoints JSON (API)

Método	Ruta	Auth	Descripción
GET	/api/me	Cookie	Devuelve el usuario autenticado, su rol y sus permisos.
GET	/api/users	Cookie	Lista todos los usuarios (solo admins).
GET	/api/audit-logs	Cookie	Lista los últimos eventos de auditoría (solo admins).
GET	/health	Ninguna	Endpoint de salud; devuelve {"status": "ok"}.

8.2. Rutas HTML principales

Método	Ruta	Descripción
GET	/login	Formulario de inicio de sesión.
POST	/login	Procesa el login (contraseña o criptográfico).
GET	/register	Formulario de autoregistro.
POST	/register	Envía la solicitud de registro.
GET	/portal	Portal del usuario autenticado.
GET	/dashboard	Panel de administración.
GET	/admin/register	Formulario de creación de usuario (admin).
POST	/admin/register	Crea un usuario nuevo.
GET	/logout	Cierra la sesión.

8.3. Rutas de gestión de usuarios (UI)

Método	Ruta	Descripción
POST	/ui/users	Crea un usuario desde el dashboard.
POST	/ui/users/{id}/status	Activa o revoca una cuenta.
POST	/ui/users/{id}/expiration	Modifica la fecha de expiración.

Método	Ruta	Descripción
POST	/ui/users/{id}/role	Cambia el rol del usuario.
POST	/ui/users/{id}/certificate	Reemite credenciales criptográficas.
POST	/ui/users/{id}/unlock	Desbloquea una cuenta bloqueada.
GET	/ui/users/{id}/certificate/view	Visualiza el certificado X.509 del usuario.
GET	/ui/users/{id}/certificate.pem	Descarga el certificado en formato PEM.
GET	/ui/users/{id}/public-key.pem	Descarga la llave pública en formato PEM.
GET	/ui/users/{id}/private-key.pem	Descarga única de la llave privada cifrada.

8.4. Rutas de beneficiarios y exportación

Método	Ruta	Descripción
POST	/ui/beneficiarios	Registra un nuevo beneficiario.
POST	/ui/beneficiarios/{id}/status	Cambia el estatus de un beneficiario.
POST	/ui/beneficiarios/{id}/delete	Elimina un beneficiario.
GET	/ui/audit-logs/export.pdf	Exporta la auditoría en PDF.
GET	/ui/users/export.pdf	Exporta la lista de usuarios en PDF.
GET	/ui/beneficiarios/export.pdf	Exporta beneficiarios en PDF.

8.5. Parámetros de ejemplo

POST /login — Login con contraseña:

Content-Type: application/x-www-form-urlencoded

identifier=admin&password=admin

POST /login — Login criptográfico:

Content-Type: multipart/form-data

identifier: coord.legal@demo.local

password: demo1234

private_key_file: (archivo private_key.pem)

certificate_file: (archivo certificate.pem)

POST /ui/users/{id}/status:

Content-Type: application/x-www-form-urlencoded

status=active&new_secret=ContraseñaSegura123

9. Flujos y Lógica de Negocio

9.1. Procesos clave

9.1.1. Ciclo de vida de una cuenta

Todo usuario inicia en estado `pending`. Un administrador lo revisa y lo activa. Si el rol requiere criptografía (Administrador o Coordinador), la activación genera:

1. Un par de llaves RSA de 2048 bits.
2. Un certificado X.509 firmado (autofirmado para admins, firmado por el admin para coordinadores).
3. La llave privada cifrada con AES-256 usando la contraseña asignada.
4. El hash PBKDF2 de la contraseña se almacena como credencial de acceso.

Los estados posibles son: `pending` → `active` → `revoked` o `expired`.

9.1.2. Autenticación criptográfica (challenge-response)

El flujo de login para roles con certificado:

1. El usuario sube `private_key.pem`, `certificate.pem` y su contraseña.
2. El servidor descifra la llave privada con la contraseña proporcionada.
3. Valida el certificado: serial, correo, vigencia y firma del emisor.
4. Genera un reto único: `login:<id>:<email>:<timestamp>`.
5. Firma el reto con la llave privada usando RSA-PSS con SHA-256.
6. Verifica la firma con la llave pública contenida en el certificado.
7. Si todo es correcto, se crea la sesión.

9.1.3. Sesión y cookies

La sesión se mantiene con una cookie firmada con HMAC mediante `URLSafeTimedSerializer` de la librería `itsdangerous`. La cookie tiene las banderas `httponly`, `samesite=lax` y una duración máxima de 8 horas.

9.1.4. Expiración automática

Al iniciar la aplicación y al listar usuarios, el servicio `ExpirationService` verifica si algún usuario activo tiene su `end_date` vencida y cambia su estado a `expired` automáticamente.

9.2. Reglas importantes

- La llave privada solo se puede descargar una vez. Después de la descarga, el servidor la elimina permanentemente.
- El administrador principal no puede ser revocado directamente; se debe usar el mecanismo de espejo.
- Al revocar un usuario con rol criptográfico, se elimina su `password_hash`.
- Al reactivar, se genera material criptográfico completamente nuevo (nuevo par de llaves, nuevo certificado, nuevo serial).

- La vigencia del certificado X.509 no se puede modificar sin reemitir. Está sellada criptográficamente en el certificado.
- Los usuarios se bloquean automáticamente tras 10 intentos fallidos de login. Solo un administrador puede desbloquearlos.

9.3. Casos críticos

- **Pérdida de llave privada:** El usuario no puede iniciar sesión. El administrador debe revocar y reactivar la cuenta para generar material nuevo.
- **Pérdida de acceso del administrador:** Se activa el administrador espejo, lo cual revoca permanentemente la cuenta principal.
- **Certificado vencido:** El sistema rechaza el login. Se genera una notificación 30 días antes del vencimiento.

10. Ejemplos de Uso

10.1. Caso 1: Crear y activar un coordinador

1. Iniciar sesión como administrador.
2. En el dashboard, hacer clic en el formulario de creación de usuario.
3. Ingresar nombre, correo, seleccionar el rol Coordinador y definir una contraseña.
4. El usuario se crea en estado pending.
5. Expandir la fila del usuario, escribir la contraseña y hacer clic en Activar.
6. El sistema genera el par de llaves RSA y el certificado X.509.
7. Hacer clic en Descargar archivo de acceso para obtener private_key.pem (descarga única).
8. Hacer clic en Descargar certificado para obtener certificate.pem.
9. Entregar ambos archivos y la contraseña al usuario por un canal seguro.

10.2. Caso 2: Iniciar sesión con certificado

1. Abrir `http://127.0.0.1:8000/login`.
2. Activar la opción de Acceso con archivos de identidad.
3. Subir private_key.pem y certificate.pem.
4. Ingresar el correo y la contraseña.
5. El sistema ejecuta la verificación criptográfica y redirige al portal.

10.3. Caso 3: Revocar un usuario

1. Desde el dashboard, expandir la fila del usuario.
2. Hacer clic en Revocar de emergencia.
3. El estado cambia a revoked y se registra en la auditoría.
4. Si el usuario intenta entrar, el servidor rechaza el login.

10.4. Caso 4: Exportar auditoría

1. En el dashboard, ir a la sección de auditoría.
2. Hacer clic en Exportar PDF.
3. Se descarga un PDF con la tabla de eventos y un hash SHA-256 de integridad al pie.

11. Manejo de Errores

11.1. Estrategia de errores

Los errores se manejan con excepciones de Python:

- Las funciones en `services.py` lanzan `ValueError` con un mensaje descriptivo cuando una operación no se puede completar (usuario no encontrado, contraseña incorrecta, certificado inválido, etc.).
- Las rutas en `main.py` capturan estas excepciones con bloques `try/except` y redirigen al dashboard o portal mostrando el mensaje de error en un banner.
- Para errores de autorización, se usa `HTTPException` con código 403.
- Para sesiones inválidas, se redirige al login con código 303.

11.2. Mensajes comunes

Mensaje	Causa
"Usuario no registrado"	El correo ingresado no existe en la base de datos.
"La cuenta no está activa: revoked"	El usuario fue revocado por un administrador.
"No se pudo abrir <code>private_key.pem</code> con esa contraseña"	La contraseña no coincide con la que se usó para cifrar la llave privada.
"El certificado no corresponde al usuario registrado"	El serial del certificado subido no coincide con el registrado.
"Cuenta bloqueada por demasiados intentos fallidos"	Se superaron los 10 intentos permitidos.
"Debes definir una nueva contraseña para el material criptográfico"	Se intenta activar un usuario criptográfico sin definir contraseña.
"La llave privada ya fue entregada"	Se intenta descargar la llave privada por segunda vez.

11.3. Registro (logs)

Todas las acciones relevantes se almacenan en la tabla `audit_logs` mediante `AuditService.log()`. Cada registro incluye:

- `event_type` — Tipo de evento (`login_success`, `login_failure`, `status_change`, `certificate_issued`, etc.).
- `action` — Acción realizada.
- `result` — `success` o `failure`.
- `actor_user_id` / `target_user_id` — Quién hizo la acción y sobre quién.
- `ip_address` y `user_agent` — Origen de la petición.
- `metadata_json` — Datos adicionales en formato JSON.

12. Pruebas

12.1. Tipos de pruebas

En la versión actual, las pruebas se realizan de forma manual siguiendo un flujo funcional completo. No se cuenta con un framework de pruebas automatizadas integrado.

12.2. Pruebas funcionales manuales

Para verificar el correcto funcionamiento del sistema:

1. Iniciar el servidor con `uvicorn app.main:app -reload`.
2. Entrar como administrador (`admin / admin`).
3. Crear un usuario con rol Coordinador, definir contraseña y activarlo.
4. Descargar los archivos `private_key.pem` y `certificate.pem`.
5. Cerrar sesión e iniciar sesión como el usuario creado usando los archivos descargados.
6. Verificar que el portal muestra "Identidad verificada con archivos de acceso".
7. Volver al administrador, revocar al usuario y confirmar que ya no puede acceder.
8. Reactivar al usuario con nueva contraseña y verificar que puede entrar con los nuevos archivos.
9. Cambiar el rol del usuario a Voluntario y verificar que ya no requiere archivos de acceso.
10. Exportar el PDF de auditoría y verificar que contiene los eventos registrados con su hash.
11. Verificar el bloqueo automático tras 10 intentos fallidos de login.

13. Despliegue

13.1. Despliegue con Docker

```
docker build -t gestor-identidades .
docker run -p 8000:8000 \
  -e ENVIRONMENT=production \
  -e DATABASE_URL=postgresql+psycopg://user:pass@host/db \
  -e SESSION_SECRET=$(openssl rand -hex 32) \
  -e SEED_DEMO_DATA=false \
  -e BOOTSTRAP_ADMIN_EMAIL=admin@tu-dominio.com \
  -e BOOTSTRAP_ADMIN_PASSWORD=clave-segura \
  gestor-identidades
```

13.2. Despliegue en Render

El archivo `render.yaml` contiene la configuración declarativa para Render:

- Servicio web tipo Docker.
- Base de datos PostgreSQL provisionada automáticamente.
- `SESSION_SECRET` generado aleatoriamente por Render.
- Endpoint de salud: `/health`.

Para desplegar, basta con conectar el repositorio de GitHub a Render y la plataforma detecta el `render.yaml` automáticamente.

13.3. Entornos

Entorno	Características
Desarrollo	SQLite, datos de demostración cargados, bypass de admin por contraseña, cookies sin Secure.
Producción	PostgreSQL, sin datos demo, <code>SESSION_SECRET</code> obligatorio, cookies Secure por HTTPS, admin bootstrap requerido.

13.4. Consideraciones

- En producción, `SESSION_SECRET` debe ser un valor aleatorio de al menos 32 bytes. Si se deja el valor por defecto, la aplicación lanza un error y no arranca.
- La base de datos PostgreSQL debe estar accesible desde el servidor. Render la provisiona en la misma red.
- Los certificados generados se almacenan en la base de datos y en el sistema de archivos (`CERTS_DIR`). En entornos efímeros (contenedores) los archivos del filesystem se pierden al reiniciar, pero la información persiste en la base de datos.
- Se recomienda ejecutar detrás de un proxy reverso con HTTPS (Render lo provee de forma nativa).

14. Seguridad

14.1. Autenticación

El sistema implementa dos mecanismos de autenticación según el rol del usuario:

Autenticación por contraseña (roles Operativo y Voluntario):

- La contraseña se almacena como hash PBKDF2 con 120,000 iteraciones de HMAC-SHA256 y una sal aleatoria de 16 bytes generada con `secrets.token_bytes()`.
- El formato almacenado es: `pbkdf2_sha256$iteraciones$sal_b64$digest_b64`.
- La verificación usa `hmac.compare_digest()` para prevenir ataques de temporización.

Autenticación criptográfica (roles Administrador y Coordinador):

- Se requieren dos archivos (`private_key.pem` y `certificate.pem`) más la contraseña.
- La llave privada está cifrada con AES-256 en formato PKCS#8.
- El certificado X.509 se valida completamente: serial, correo, vigencia y firma del emisor.
- Se genera un reto con timestamp que se firma con RSA-PSS (SHA-256) para verificar la posesión de la llave privada.

14.2. Manejo de datos sensibles

- Las llaves privadas de los usuarios se cifran con AES-256 antes de almacenarse y se eliminan del servidor después de la primera descarga.
- La llave privada de la CA y la del administrador firmante se almacenan en la tabla `system_secrets` sin cifrado adicional (el acceso a la base de datos debe estar protegido a nivel de infraestructura).
- Las cookies de sesión se firman con HMAC y llevan las banderas `httponly` y `samesite=lax` para mitigar XSS y CSRF.
- Los tokens de sesión expiran automáticamente a las 8 horas.
- Los reportes PDF llevan un hash SHA-256 calculado sobre los datos del reporte para detectar manipulaciones.

14.3. Buenas prácticas implementadas

- Escape de HTML con `html.escape()` para prevenir XSS en las vistas.
- Bloqueo automático de cuentas tras 10 intentos fallidos.
- Contraseñas nunca se almacenan en texto claro.
- Descarga única de llave privada: el material se elimina permanentemente del servidor después de la primera entrega.
- En producción, la cookie de sesión lleva la bandera `Secure` (solo HTTPS).
- Validación de entradas en los formularios antes de procesarlas.
- Separación de permisos por rol a nivel de base de datos.

15. Guía de Contribución

15.1. Convenciones de código

- Código en Python 3.11+ con type hints.
- Nombres de variables y funciones en `snake_case`.
- Clases con `PascalCase`.
- Constantes en `UPPER_SNAKE_CASE`.
- El HTML se genera con f-strings en `main.py`. No se usan plantillas externas.
- Los servicios se organizan como clases con métodos estáticos o de clase en `services.py`.

15.2. Flujo de trabajo con Git

1. Hacer fork del repositorio.
2. Clonar el fork: `git clone <url-del-fork>`.
3. Crear una rama descriptiva: `git checkout -b feature/mi-cambio`.
4. Leer `COMO_FUNCIONA_EL_PROYECTO.md` y la documentación en `docs/` para entender la arquitectura.
5. Hacer los cambios y probar que el servidor arranca sin errores.
6. Hacer commit con mensajes claros en español.
7. Abrir un Pull Request describiendo qué se modificó y por qué.

15.3. Cómo agregar funcionalidades

- **Nuevo endpoint:** Agregar la ruta en `main.py` siguiendo el patrón de las rutas existentes. Si requiere lógica de negocio, encapsularla en un servicio en `services.py`.
- **Nuevo modelo:** Definir la clase ORM en `models.py` heredando de `Base`. SQLAlchemy la crea automáticamente al arrancar.
- **Nuevo permiso:** Agregar la tupla (`resource`, `action`) en `ROLE_DEFINITIONS` dentro de `services.py` para el rol correspondiente.
- **Cambios criptográficos:** Revisar cuidadosamente `CertificateService` y `SignatureLoginService` antes de modificar cualquier parámetro. Un cambio en el esquema de firma invalida todos los certificados existentes.

16. Problemas Conocidos

16.1. Bugs actuales

Al tratarse de la primera versión del sistema, no se han reportado bugs pendientes de corrección.

16.2. Limitaciones

- **Hosting:** En plataformas con filesystem efímero (como Render en plan gratuito), los archivos generados en disco se pierden al reiniciar el servicio. El material criptográfico persiste en la base de datos, pero los archivos de la CA en `generated/certs/` deben regenerarse.
- **Correo electrónico:** No se implementa envío de correos para notificaciones ni recuperación de contraseña. Las notificaciones solo se muestran dentro del sistema.
- **Escalabilidad:** La arquitectura monolítica con HTML generado en el servidor no es ideal para aplicaciones de alto tráfico. Para escalar se recomendaría separar el frontend en una SPA.
- **Base de datos en desarrollo:** SQLite no soporta conexiones concurrentes de escritura, lo cual puede causar bloqueos si múltiples usuarios operan simultáneamente en desarrollo.
- **Sin pruebas automatizadas:** La validación del sistema depende de pruebas manuales. Se recomienda agregar pruebas unitarias con `pytest` en futuras iteraciones.

17. FAQ Técnica

El servidor no arranca y dice que SESSION_SECRET debe definirse.

Esto ocurre cuando `ENVIRONMENT=production` pero `SESSION_SECRET` tiene el valor por defecto. Hay que definir un secreto aleatorio en el archivo `.env` o como variable de entorno. Se puede generar uno con:

```
python -c 'import secrets; print(secrets.token_hex(32))'
```

Al instalar dependencias sale un error con psycopg.

El paquete `psycopg[binary]` requiere compilación nativa en algunos sistemas. Si no se va a usar PostgreSQL (solo desarrollo con SQLite), se puede instalar sin ese paquete o ignorar el error.

No puedo descargar la llave privada de un usuario.

La llave privada solo se puede descargar una vez. Si ya fue descargada, hay que reemitir las credenciales desde la sección de Credencial del usuario en el dashboard.

Cambié el SESSION_SECRET y ahora nadie puede entrar.

Al cambiar el secreto, todas las cookies de sesión existentes se invalidan porque la firma HMAC ya no coincide. Los usuarios deben iniciar sesión de nuevo.

La aplicación dice “No hay usuarios iniciales configurados”.

Esto pasa cuando `SEED_DEMO_DATA=false` y no se definieron `BOOTSTRAP_ADMIN_EMAIL` ni `BOOTSTRAP_ADMIN_PASSWORD`. Hay que definir ambas variables para crear el administrador inicial.

¿Cómo regenero el certificado de la CA?

La CA se genera automáticamente al iniciar la aplicación si no existe. Para regenerar, hay que eliminar las entradas `ca_private_key_pem` y `ca_certificate_pem` de la tabla `system_secrets` y reiniciar el servidor. Esto invalida todos los certificados de coordinadores existentes.

¿Qué pasa si se pierde la base de datos?

Se pierde toda la información: usuarios, certificados, auditoría y beneficiarios. Se recomienda hacer respaldos periódicos de la base de datos, especialmente en producción.

18. Referencias

- FastAPI — Documentación oficial: <https://fastapi.tiangolo.com>
- SQLAlchemy 2.0 — Documentación: <https://docs.sqlalchemy.org/en/20/>
- Librería cryptography de Python: <https://cryptography.io/en/latest/>
- itsdangerous — Firma de datos: <https://itsdangerous.palletsprojects.com>
- fpdf2 — Generación de PDF: <https://py-pdf.github.io/fpdf2/>
- uvicorn — Servidor ASGI: <https://www.uvicorn.org>
- Docker — Documentación: <https://docs.docker.com>
- Render — Plataforma de despliegue: <https://render.com/docs>
- Python 3.11 — Descarga: <https://www.python.org/downloads/>